

✓ Challenge 4 — Imaging

The Chest X-Ray (CXR) is the most common image used in the NHS. In 2023/2024, the NHS reported 22 million CXRs requested and reported. Of these, 20% were normal, with a high margin of confidence. That's still 4 million CXRs that have utilised a precious human resource.

Mission

Deploy and evaluate a pre-trained medical imaging AI for chest X-ray classification:

1. Load **TorchXRayVision** — a DenseNet-121 pre-trained on 100,000+ real clinical chest X-rays
2. Select the best pre-trained weight set for your task through structured evaluation
3. Map the model's 18 clinical outputs to your 4-class challenge task
4. Benchmark against the HuggingFace chest X-ray test set — report accuracy, sensitivity, and a confusion matrix
5. Visualise model reasoning using **GradCAM** (no fine-tuning required)
6. Combine with GPT-4o-mini to generate structured radiological reports
7. Add clinical guardrails and deploy as a live image upload app

Why This Matters

Using AI to automate CXR review and report is a high-yield measure that can release clinicians to focus on high-touch (intervention) and high complexity (second read cancer screening) tasks.

Why No Fine-Tuning?

Cost! But also: Fine-tuning a general use pre-trained model from scratch on a small dataset is often **worse** than using a model already trained on a large, curated clinical dataset.

TorchXRayVision's DenseNet-121 was trained by researchers at Stanford, Montréal and MIT on:

Dataset	Images	Source
NIH ChestX-ray14	112,000	US National Institutes of Health
CheXpert	224,000	Stanford University Medical Center
MIMIC-CXR	370,000	MIT / Beth Israel Deaconess Medical Center
PadChest	160,000	Hospital San Juan, Spain
RSNA Pneumonia	30,000	Radiological Society of North America

Your task today is the job of a clinical AI team in the real world: **select, evaluate, explain, and safely deploy a pre-trained model.**

Good news: No GPU required. TorchXRayVision runs on CPU in seconds per image. You can use the standard Colab runtime.

✓ Setup — API Key and OpenAI Client

Start coding or [generate](#) with AI.

```
from google.colab import userdata
import os, json
from openai import OpenAI

client = OpenAI(api_key=userdata.get('OPENAI_API_KEY'))
print('OK OpenAI client ready.')
```

OK OpenAI client ready.

✓ Step 1 — Install Libraries

```
!pip install -q torchxrayvision scikit-image datasets Pillow scikit-learn

import torch
import torchxrayvision as xrv
import skimage
import torchvision
import torchvision.transforms as T
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
from datasets import load_dataset
from collections import Counter
print('OK All libraries loaded.')
print(f'TorchXRayVision version: {xrv.__version__}')
print(f'PyTorch: {torch.__version__}')
```

✓ Step 2 — Load the Test Dataset

We use the same HuggingFace chest X-ray dataset as before, but this time we only need the **test split** — there is nothing to train, so we go straight to evaluation.

In this task, we are evaluating the model on data it has **never seen**, from a different source than its training data. This is called **out-of-distribution evaluation** and it tells you far more about real-world clinical performance than validation accuracy ever can.

We are also going to sample just the minimum number of images to deliver on the task (200).

```
import random
from datasets import load_dataset
from collections import Counter

print('Loading chest X-ray test set from HuggingFace...')
# The original dataset 'keremberke/chest-xray-classification' is no longer available
# due to deprecated script loading. We are switching to a compatible dataset
dataset = load_dataset('KVG-DataScience/chest_xray_dataset')

# We only need the test set for evaluation
test_ds_full = dataset['test']

# Randomly sample a smaller subset
NUM_SAMPLES = 200 # Adjust this number as needed for faster execution
if len(test_ds_full) > NUM_SAMPLES:
    test_ds = test_ds_full.shuffle(seed=42).select(range(NUM_SAMPLES))
else:
    test_ds = test_ds_full

# The new dataset 'KVG-DataScience/chest_xray_dataset' has different labels
# It typically has 'NORMAL' and 'PNEUMONIA' classes.
LABELS = test_ds.features['label'].names
print(f'Test images: {len(test_ds)}')
print(f'Classes: {LABELS}')

counts = Counter(test_ds['label'])
print('\nTest set distribution:')
for i, l in enumerate(LABELS):
    print(f'  {l:20s}: {counts.get(i,0):>4} images')
```

Step 3 — Load TorchXRayVision Pre-Trained Model

TorchXRayVision offers several pre-trained weight sets. The key difference is **which clinical dataset each model was trained on**, which directly affects which pathologies it detects well and which populations it generalises to.

Available Weight Sets

Weights	Training Data	Best for
<code>densenet121-res224-all</code>	All datasets combined	Best overall generalisability
<code>densenet121-res224-rsna</code>	RSNA Pneumonia Challenge	Highest sensitivity for pneumonia
<code>densenet121-res224-nih</code>	NIH ChestX-ray14 (112k images)	Broad pathology coverage
<code>densenet121-res224-chex</code>	CheXpert (Stanford)	High-quality PA film performance
<code>densenet121-res224-mimic_nb</code>	MIMIC-CXR (MIT/BIDMC)	Large US hospital population

We start with `densenet121-res224-all` (trained on all datasets) — the most robust choice for an unknown test set. You can compare weight sets in Step 6.

✓ Step 4 — Map Model Outputs to the Challenge Classes

The TorchXRayVision model outputs 18 pathology scores. Our challenge has 4 classes. We need to map between them.

This is a genuine clinical informatics task: real-world models rarely use exactly the same label taxonomy as your evaluation dataset. Understanding how to bridge this gap — and the uncertainty it introduces — is part of responsible AI deployment.

Mapping Strategy

Challenge Class	TorchXRayVision Output(s) Used
Pneumonia	<code>Pneumonia</code> + <code>Consolidation</code> (consolidation is the radiological correlate of pneumonia)
COVID-19	<code>Consolidation</code> + <code>Infiltration</code> (COVID-19 was not in training data; see note below)
Pleural Effusion	<code>Effusion</code>
Normal	All pathology scores below threshold

⚠ COVID-19 note: TorchXRayVision models were trained largely on pre-pandemic data. COVID-19 as a labelled class does not exist in the training data. The model may detect the radiological *features* of COVID-19 (bilateral consolidation, ground-glass opacity) without knowing the aetiological label. This is a real limitation and must be declared in your model card.

```
WEIGHTS = 'densenet121-res224-all'    # change this in Step 6 to compa

print(f'Loading TorchXRayVision: {WEIGHTS}...')
model = xrv.models.DenseNet(weights=WEIGHTS)
model.eval()

print(f'OK Model loaded.')
print(f'Architecture:   DenseNet-121')
print(f'Input size:      224 × 224 (greyscale)')
print(f'Output:           {len(model.pathologies)} pathologies (sigmoid
print(f'\nAll pathologies the model can detect:')
for i, p in enumerate(model.pathologies):
    print(f' {i+1:2d}. {p}')
```

```
# Map TorchXRayVision pathology indices to challenge classes
def get_pathology_idx(name):
    """Get index of a named pathology in the model's output vector."""
    return model.pathologies.index(name) if name in model.pathologies

PNEUMONIA_IDX = get_pathology_idx('Pneumonia')
```

```

CONSOLID_IDX = get_pathology_idx('Consolidation')
INFILTR_IDX = get_pathology_idx('Infiltration')
EFFUSION_IDX = get_pathology_idx('Effusion')

print('Pathology index mapping:')
print(f'  Pneumonia (index {PNEUMONIA_IDX})')
print(f'  Consolidation (index {CONSOLID_IDX})')
print(f'  Infiltration (index {INFILTR_IDX})')
print(f'  Effusion (index {EFFUSION_IDX})')

# Classification threshold – tune this in Step 5
THRESHOLD = 0.30 # sigmoid output above this = positive detection

def predict_challenge_class(probs, threshold=THRESHOLD):
    """Map 18-pathology sigmoid outputs to one of 'NORMAL' or 'PNEUMONIA'
    # Combine scores for pneumonia and consolidation, as these are the most relevant
    pneumonia_score = max(probs[PNEUMONIA_IDX], probs[CONSOLID_IDX])

    # For a binary classification task (NORMAL/PNEUMONIA)
    if pneumonia_score >= threshold:
        # If the combined pneumonia score is above the threshold, classify as PNEUMONIA
        return 'PNEUMONIA', pneumonia_score, {'PNEUMONIA': pneumonia_score}
    else:
        # Otherwise, classify as NORMAL
        return 'NORMAL', 1 - pneumonia_score, {'PNEUMONIA': pneumonia_score}

print('\nOK predict_challenge_class() ready.')
print(f'Detection threshold: {THRESHOLD}')

```

✓ Step 5 — Preprocess and Run Inference on Test Set

TorchXRayVision has its own preprocessing pipeline:

- Images must be **greyscale** (single channel)
- Pixel values must be **normalised to [-1024, 1024]** (DICOM range)
- Images are **centre-cropped and resized to 224×224**

This is intentionally different from standard ImageNet preprocessing — chest X-rays have their own intensity distribution and preprocessing must respect the clinical image standard.

```

# TorchXRayVision preprocessing pipeline
transform = T.Compose([
    xrv.datasets.XRayCenterCrop(),
    xrv.datasets.XRayResizer(224),
])

def preprocess_image(pil_img):
    """Convert PIL image to TorchXRayVision tensor."""
    img = np.array(pil_img.convert('L')) # greyscale
    img = xrv.datasets.normalize(img, 255) # [0,255] → [-1024,1024]

```

```

img = img[None, ...] # add channel dim → (
img = transform(img) # crop + resize
return torch.from_numpy(img).unsqueeze(0) # add batch dim → (1,

# Run inference on entire test set
print('Running inference on test set...')

y_true, y_pred, all_probs = [], [], []

for i, sample in enumerate(test_ds):
    img_tensor = preprocess_image(sample['image'])
    with torch.no_grad():
        probs = torch.sigmoid(model(img_tensor))[0].numpy() # 18-d p
    pred_class, score, _ = predict_challenge_class(probs)
    y_true.append(LABELS[sample['label']])
    y_pred.append(pred_class)
    all_probs.append(probs)
    if (i+1) % 50 == 0:
        print(f' {i+1}/{len(test_ds)} done...')

print(f'\nOK Inference complete on {len(test_ds)} images.')

```

✓ Step 6 — Evaluate: Accuracy, Sensitivity, Confusion Matrix

Now we assess performance against the ground-truth test labels. The judges will score your model on a **10-image live test set** at demo time. This full evaluation tells you what to expect.

```

from sklearn.metrics import (classification_report, confusion_matrix,
                             ConfusionMatrixDisplay, accuracy_score

# Overall accuracy
acc = accuracy_score(y_true, y_pred)
print(f'Overall Accuracy: {acc*100:.1f}%')

# Per-class report
print('\n--- Per-Class Report ---')
print(classification_report(y_true, y_pred, labels=LABELS, digits=3))

# Pneumonia sensitivity – the safety-critical metric
from sklearn.metrics import recall_score
pneumonia_recall = recall_score(
    [1 if y=='PNEUMONIA' else 0 for y in y_true],
    [1 if y=='PNEUMONIA' else 0 for y in y_pred]
)
print(f'🚨 Pneumonia sensitivity: {pneumonia_recall*100:.1f}% (judg
print(f' A missed pneumonia = 0 pts for that image on demo day.')

# Confusion matrix
fig, ax = plt.subplots(figsize=(7, 6))

```

```
ConfusionMatrixDisplay.from_predictions(y_true, y_pred, labels=LABELS)
ax.set_title(f'Confusion Matrix - TorchXRayVision ({WEIGHTS})', fonts
plt.tight_layout()
plt.savefig('confusion_matrix.png', dpi=120)
plt.show()
print('Saved: confusion_matrix.png')
```

Step 6b — Model Comparison: Which Weight Set Performs Best?

This is the core learning task replacing fine-tuning. Rather than training your own model, you are **selecting the best pre-trained model** for this specific clinical task.

Run the cell below to compare all available weight sets on the same test set. This teaches a crucial real-world skill: **off-the-shelf models vary significantly in performance depending on what clinical population they were trained on.**

```
WEIGHT_SETS = [
    'densenet121-res224-all',
    'densenet121-res224-rsna',
    'densenet121-res224-nih',
    'densenet121-res224-chex',
    'densenet121-res224-mimic_nb',
]

results = {}
for w in WEIGHT_SETS:
    print(f'Testing {w}...')
    try:
        m = xrv.models.DenseNet(weights=w)
        m.eval()
        yt, yp = [], []
        for sample in test_ds:
            img_tensor = preprocess_image(sample['image'])
            with torch.no_grad():
                probs = torch.sigmoid(m(img_tensor))[0].numpy()
            # Re-index pathologies for this weight set
            p_idx = m.pathologies.index('Pneumonia') if 'Pneumonia' in m.pathologies else -1
            c_idx = m.pathologies.index('Consolidation') if 'Consolidation' in m.pathologies else -1
            e_idx = m.pathologies.index('Effusion') if 'Effusion' in m.pathologies else -1
            i_idx = m.pathologies.index('Infiltration') if 'Infiltration' in m.pathologies else -1
            scores = {}
            if p_idx and c_idx: scores['Pneumonia'] = max(probs[p_idx], probs[c_idx])
            if c_idx and i_idx: scores['COVID-19'] = max(probs[c_idx], probs[i_idx])
            if e_idx: scores['Pleural Effusion'] = probs[e_idx]
            pred = max(scores, key=scores.get) if scores else -1
            yt.append(LABELS[sample['label']])
            yp.append(pred)
        acc = accuracy_score(yt, yp)
        pneu_recall = recall_score(
```

```

        [1 if y=='Pneumonia' else 0 for y in yt],
        [1 if y=='Pneumonia' else 0 for y in yp]
    )
    results[w] = {'accuracy': acc, 'pneumonia_sensitivity': pneu_
    print(f' Accuracy: {acc*100:.1f}% | Pneumonia sensitivity:
except Exception as e:
    print(f' Skipped: {e}')
    results[w] = None

print('\n--- Summary ---')
print(f'{"Weights":<35} {"Accuracy":>10} {"Pneu. Recall":>14}')
print('-'*62)
for w, r in results.items():
    if r:
        print(f'{w:<35} {r["accuracy"]*100:>9.1f}% {r["pneumonia_sens

```

✓ Step 7 — Generate Radiological Reports with GPT-4o-mini

The DenseNet classifies across 18 pathologies. GPT-4o-mini articulates those findings as a structured consultant-radiologist report.

Unlike the fine-tuning approach, we now pass the **full 18-pathology probability vector** to GPT-4o-mini — not just the top prediction. This gives the language model richer clinical context and allows it to note secondary findings (e.g. "Effusion score is borderline at 0.28, radiologist attention recommended").

```

RADIOLOGY_SYS = (
    'You are a consultant radiologist generating a structured chest X-
    'You will receive a vector of probability scores (0-1) for 18 rad
    'from a pre-trained DenseNet-121 model (TorchXRayVision). '
    'Use this exact format: '
    'CHEST X-RAY REPORT | PRIMARY FINDING | SECONDARY FINDINGS | CLIN
    'Note any borderline scores (0.20-0.40) as requiring clinical att
    'Do not exceed 5 sentences. Be clinically precise.'
)

def classify_and_report(image_path_or_pil, title='Chest X-Ray'):
    """Full pipeline: preprocess → DenseNet inference → GPT-4o-mini r
    if isinstance(image_path_or_pil, str):
        pil_img = Image.open(image_path_or_pil)
    else:
        pil_img = image_path_or_pil

    img_tensor = preprocess_image(pil_img)
    with torch.no_grad():
        probs = torch.sigmoid(model(img_tensor))[0].numpy()

    # Full 18-pathology probability dict
    full_findings = {p: round(float(v), 3)
                     for p, v in zip(model.pathologies, probs)}

```

```

# Challenge-class prediction
pred_class, confidence, class_scores = predict_challenge_class(pr

# Build GPT prompt with full findings
prompt = (
    f'18-pathology probability scores from DenseNet-121: {full_fi
    f'Top challenge-class prediction: {pred_class} '
    f'(composite confidence {confidence:.3f}).\n'
    'Generate a structured radiological report.'
)
r = client.chat.completions.create(
    model='gpt-4o-mini',
    messages=[
        {'role': 'system', 'content': RADIOLOGY_SYS},
        {'role': 'user', 'content': prompt}
    ],
    temperature=0.1,
    max_tokens=300
)
report = r.choices[0].message.content
return pred_class, confidence, full_findings, report

print('OK classify_and_report() ready.')
```

✓ Step 8 — XAI: Explain the Model with GradCAM

Why does the model think it sees pneumonia?

GradCAM (Gradient-weighted Class Activation Mapping) is the standard XAI method for CNN architectures like DenseNet-121. It computes gradients flowing back from the predicted class score into the final convolutional layer, then visualises which spatial regions activated most strongly. This produces a heatmap overlaid on the original X-ray.

Why GradCAM is Clinically Important

A model that correctly classifies "Pneumonia" but attends to the *left shoulder* instead of the *right lower lobe* is not clinically trustworthy — it has learned a spurious correlation (perhaps position of a medical device) rather than true pneumonic consolidation.

GradCAM lets a radiologist verify that the model is looking in the right place.

XAI Method — for your Slide 2

Method	Architecture	Output	Status
GradCAM	DenseNet, ResNet, VGG (CNN)	Class activation heatmap	✅ This notebook
Attention Rollout	Vision Transformer	Patch attention weights	For ViT architectures
LIME	Any	Superpixel attribution	Model-agnostic but slow

Method	Architecture	Output	Status
SHAP	Any	Pixel-level attribution	Computationally expensive

```

import torch.nn.functional as F
from PIL import Image

def gradcam(model, img_tensor, target_class_name):
    """
    Compute GradCAM heatmap for a target pathology class.
    Works with TorchXRayVision DenseNet-121.
    """
    target_idx = model.pathologies.index(target_class_name)
    gradients = []
    activations = []

    # Hook: capture gradients at final convolutional layer
    def save_gradient(grad): gradients.append(grad)
    def save_activation(module, input, output):
        activations.append(output)
        output.register_hook(save_gradient)

    # Register hooks on final dense block
    handle = model.features.denseblock4.register_forward_hook(save_ac

    # Forward pass
    output = torch.sigmoid(model(img_tensor))
    handle.remove()

    # Backward pass from target class
    model.zero_grad()
    output[0, target_idx].backward(retain_graph=True)

    # GradCAM computation
    pooled_grads = gradients[0].mean(dim=[0, 2, 3]) # global av
    act = activations[0][0] # (C, H, W)
    for i in range(act.shape[0]):
        act[i, :, :] *= pooled_grads[i]
    heatmap = act.mean(dim=0).detach().numpy()
    heatmap = np.maximum(heatmap, 0) # ReLU
    if heatmap.max() > 0:
        heatmap /= heatmap.max() # normali
    return heatmap

def display_gradcam(image_path_or_pil, target='Pneumonia'):
    """Classify, compute GradCAM, and display a 3-panel visualisation
    if isinstance(image_path_or_pil, str):
        pil_img = Image.open(image_path_or_pil).convert('L')
    else:
        pil_img = image_path_or_pil.convert('L')

    img_tensor = preprocess_image(pil_img)

    # Run GradCAM
    if target not in model.pathologies:

```

```

        print(f'Warning: {target} not in model pathologies. Using Pneumonia')
        target = 'Pneumonia'
    heatmap = gradcam(model, img_tensor, target)

    # Upsample heatmap to original image size
    heatmap_resized = np.array(
        Image.fromarray((heatmap * 255).astype(np.uint8)).resize(pil_img.size) / 255.0)

    fig, axes = plt.subplots(1, 3, figsize=(15, 5))
    orig = np.array(pil_img)

    axes[0].imshow(orig, cmap='gray'); axes[0].set_title('Original X-ray')
    axes[1].imshow(heatmap_resized, cmap='jet'); axes[1].set_title(f'Heatmap for {target}')
    axes[2].imshow(orig, cmap='gray')
    axes[2].imshow(heatmap_resized, cmap='jet', alpha=0.45)
    axes[2].set_title('Overlay - What the model attended to'); axes[2].set_ylabel('')

    plt.suptitle(f'XAI: GradCAM for {target} - DenseNet-121 ({WEIGHTS})')
    plt.tight_layout()
    plt.savefig('gradcam_output.png', dpi=120)
    plt.show()
    print('Saved: gradcam_output.png')
    print()
    print('A radiologist should verify the heatmap attends to:')
    print('  Pneumonia          → lower lobe consolidation / airspace opacities')
    print('  Effusion              → costophrenic angle / fluid meniscus')
    print('  Consolidation         → lobar or segmental opacity')
    print('  Normal                 → no focal region should strongly dominate')

    print('OK display_gradcam() ready.')
    print('Usage: display_gradcam(pil_image, target="Pneumonia")')

```

✓ Step 9 — Clinical Guardrails

The model outputs 18 continuous probability scores — not hard binary decisions. A safe clinical deployment must define explicit rules for when to escalate, when to flag uncertainty, and when to refuse a "normal" verdict.

Evidence Base

The following thresholds are grounded in:

- Rajpurkar et al. (2017) CheXNet: radiologist-level pneumonia detection requires recall ≥ 0.90
- NHS England AI imaging guidance: human-in-the-loop required for all AI-assisted reporting
- TorchXRayVision validation studies: all-weights model achieves AUC ~0.77–0.82 on external data

```

PNEUMONIA_HARD_FLAG_THRESHOLD = 0.20    # any score above this → manda
UNCERTAINTY_THRESHOLD          = 0.25    # composite confidence below th
NORMAL_SAFETY_THRESHOLD        = 0.70    # Normal only returned if conf

def apply_guardrails(pred_class, confidence, full_findings):
    """Apply clinical safety rules to model output."""
    alerts = []

    # Rule 1: Pneumonia hard flag (any above minimum threshold)
    pneu_raw = max(
        full_findings.get('Pneumonia', 0),
        full_findings.get('Consolidation', 0)
    )
    if pneu_raw >= PNEUMONIA_HARD_FLAG_THRESHOLD:
        alerts.append(f'⚠️ EXPEDITED REVIEW – Pneumonia/Consolidation detected (threshold {PNEUMONIA_HARD_FLAG_THRESHOLD})')

    # Rule 2: Uncertainty flag
    if confidence < UNCERTAINTY_THRESHOLD:
        alerts.append(f'⚠️ UNCERTAIN – Max composite score {confidence} < {UNCERTAINTY_THRESHOLD} – Senior Radiologist review recommended')

    # Rule 3: Normal safety gate
    if pred_class == 'Normal' and confidence < NORMAL_SAFETY_THRESHOLD:
        pred_class = 'Indeterminate'
        alerts.append(f'⚠️ Normal withheld – confidence {confidence} < {NORMAL_SAFETY_THRESHOLD} safety threshold')

    # Rule 4: Secondary finding alert (borderline scores 0.20–0.40)
    borderline = [(p, v) for p, v in full_findings.items() if 0.20 <= v <= 0.40 and p not in ['Pneumonia', 'Consolidation']]
    if borderline:
        bl_str = ', '.join(f'{p} ({v:.2f})' for p, v in sorted(borderline, key=lambda x: x[1]))
        alerts.append(f'ℹ️ Borderline secondary findings: {bl_str}')

    return pred_class, alerts

print('OK Clinical guardrails defined.')
print(f'Pneumonia flag threshold:      ≥ {PNEUMONIA_HARD_FLAG_THRESHOLD}')
print(f'Uncertainty threshold:          < {UNCERTAINTY_THRESHOLD}')
print(f'Normal safety gate:               ≥ {NORMAL_SAFETY_THRESHOLD}')

```

✓ Step 10 — Format the Report as a Clinical Card

```

from IPython.display import HTML, display

URGENCY_COLOURS = {
    'Normal': ('#1a5c3a', '#e6f4eb'),
    'Pneumonia': ('#8b1a1a', '#fce8e8'),
    'COVID-19': ('#7a5200', '#fff3cd'),
    'Pleural Effusion': ('#003e74', '#e8eef5'),
}

```

```

'Indeterminate':    ('#4e5d7a', '#f2ede4'),
}

def display_radiology_card(image_path_or_pil, title='Chest X-Ray Revi
    """Full pipeline with formatted HTML output."""
    pred_class, confidence, full_findings, report = classify_and_repo
    pred_class, alerts = apply_guardrails(pred_class, confidence, ful

    fg, bg = URGENCY_COLOURS.get(pred_class, ('#1a1a2e', '#f2ede4'))
    border = '#8b1a1a' if alerts else fg

    alert_html = ''.join(
        f'<div style="background:#fce8e8;border-left:4px solid #8b1a1a;
        f'padding:10px 14px;margin-bottom:8px;border-radius:0 4px 4px
        f'font-weight:700;color:#8b1a1a">{a}</div>'
        for a in alerts
    )

    sections = [s.strip() for s in report.split('|')]
    labels_ = ['Report', 'Primary Finding', 'Secondary Findings', 'Clin
    section_html = ''.join(
        f'<tr><td style="padding:8px 14px;font-weight:700;color:{fg};
        f'white-space:nowrap;vertical-align:top;width:180px">'
        f'{labels_[i]} if i < len(labels_) else ""</td>'
        f'<td style="padding:8px 14px;color:#1a1a2e">{sec}</td></tr>'
        for i, sec in enumerate(sections)
    )

    # Top 6 pathology probability bars
    top6 = sorted(full_findings.items(), key=lambda x: -x[1])[:6]
    bars = ''.join(
        f'<div style="display:flex;align-items:center;gap:8px;margin-l
        f'<span style="width:160px;font-size:12px;color:#4e5d7a">{p}<
        f'<div style="flex:1;background:#e8eef5;border-radius:3px;hei
        f'<div style="width:{v*100:.1f}%;background:{fg};height:14px;|
        f'<span style="width:40px;font-size:12px;font-weight:700;colo
        f'</div>'
        for p, v in top6
    )

    html = f'''
<div style="border:2px solid {border};border-radius:8px;overflow:|
    font-family:'Segoe UI',Calibri,Arial,sans-serif;max-w
    <div style="background:#002147;padding:14px 20px;display:flex;
        align-items:center;justify-content:space-between">
        <div>
            <div style="font-size:11px;letter-spacing:.14em;text-transf
                color:#d4ae4a;margin-bottom:4px">
                Oxford Clinical AI · DenseNet-121 ({WEIGHTS})
            </div>
            <div style="font-size:18px;font-weight:700;color:#fff">{tit
            </div>
        <div style="text-align:right">
            <div style="font-size:26px;font-weight:700;color:{fg};backg
                padding:6px 16px;border-radius:4px">{pred_class

```

```

        <div style="font-size:12px;color:rgba(255,255,255,.55);margin-bottom:10px">
            Confidence {confidence*100:.1f}%
        </div>
    </div>
</div>
<div style="padding:14px 18px;background:#fff">
    {alert_html}
    <table style="width:100%;border-collapse:collapse;margin-bottom:10px">
        <tr>
            <td style="width:60%;padding:5px 10px">
                {section_html}
            </td>
            <td style="width:40%;padding:5px 10px">
                {bars}
            </td>
        </tr>
    </table>
    <div style="border-top:1px solid #d4d9e3;padding-top:12px">
        <div style="font-size:12px;font-weight:700;letter-spacing:.5px;color:#4e5d7a;margin-bottom:8px">Top Pathology Findings</div>
        {top_pathology_html}
    </div>
</div>
<div style="background:#f2ede4;padding:8px 18px;font-size:11px;border-top:1px solid #d4d9e3">
    ⚠ For radiologist review only · Not for autonomous clinical decisions
    TorchXRayVision DenseNet-121 · GPT-4o-mini · Oxford Clinical Deep Learning
</div>
</div>'''
display(HTML(html))
return pred_class, confidence, alerts

print('OK display_radiology_card() ready.')
```

✓ Example of a radiology report card

```

# Save an example image from the test_ds to a temporary file
example_pil_img = test_ds[0]['image']
example_image_path = 'temp_cxr_example.png'
example_pil_img.save(example_image_path)

# Run inference using the image path
print(f"Running inference for image: {example_image_path}")
display_radiology_card(example_image_path, title='Test Inference from Image')
```

✓ Step 11 — Live Image Upload App (Demo Day Interface)

```

import ipywidgets as widgets
from IPython.display import display, clear_output

upload_btn = widgets.FileUpload(accept='.jpg,.jpeg,.png', multiple=False,
                                description='Upload CXR')
analyse_btn = widgets.Button(description='Analyse', button_style='primary',
                              icon='search', layout=widgets.Layout(width='100px'))
gradcam_btn = widgets.Button(description='GradCAM', button_style='info',
                              icon='eye', layout=widgets.Layout(width='100px'))
status      = widgets.Label('Upload a chest X-ray to begin.')
```

```

output_area = widgets.Output()
current_img = [None]

def on_analyse(b):
    with output_area:
        clear_output()
        if not upload_btn.value:
            status.value = 'Please upload an image first.'; return
        content = list(upload_btn.value.values())[0]['content']
        from io import BytesIO
        pil_img = Image.open(BytesIO(content)).convert('L')
        current_img[0] = pil_img
        status.value = 'Analysing...'
        try:
            pred, conf, alerts = display_radiology_card(pil_img, title)
            status.value = f'{pred} ({conf*100:.1f}%) - {"⚠️ Alerts r
        except Exception as e:
            status.value = f'Error: {e}'

def on_gradcam(b):
    with output_area:
        if current_img[0] is None:
            status.value = 'Analyse an image first.'; return
        status.value = 'Computing GradCAM...'
        display_gradcam(current_img[0], target='Pneumonia')
        status.value = 'GradCAM complete.'

analyse_btn.on_click(on_analyse)
gradcam_btn.on_click(on_gradcam)

print('=== Oxford Clinical AI Hackathon, Challenge 4 – Chest X-Ray Cl
print ("Supported formats are .jpg, .jpg, .png")
display(widgets.VBox([
    widgets.HBox([upload_btn, analyse_btn, gradcam_btn]),
    status,
    output_area
]))

```

Step 12 — Model Card (Domain 2: Explainability Slide)

Complete this for your demo day Slide 2. Every field will be probed.

Architecture

Field	Detail
Model	TorchXRyVision DenseNet-121
Weights used	<code>densenet121-res224-all</code> (trained on all datasets)
Architecture	DenseNet-121 with 121 dense layers and skip connections
Input	Greyscale X-ray, normalised to [-1024, 1024], resized 224×224
Output	18 pathology sigmoid scores (multi-label, not mutually exclusive)
Parameters	~7 million

Field	Detail
Inference speed	~0.5s per image on CPU

Training Data (Data Provenance)

Dataset	Images	Institution
NIH ChestX-ray14	112,000	US National Institutes of Health
CheXpert	224,000	Stanford University Medical Center
MIMIC-CXR	370,000	MIT / Beth Israel Deaconess Medical Center
PadChest	160,000	Hospital San Juan, Spain
RSNA Pneumonia	30,000	Radiological Society of North America

Design Choices

Choice	Rationale
Pre-trained rather than fine-tuned	Model trained on 900k+ real clinical images outperforms a ViT fine-tuned on 1
all-weights	Best generalisation to unknown test distribution
Threshold 0.30	Balances specificity and recall; lower increases sensitivity at cost of false posi
GradCAM over Attention Rollout	DenseNet is a CNN; GradCAM is the standard and most clinically interpretable

Known Limitations (*Be honest — judges reward this*)

- COVID-19 is not a labelled class in training data; model detects radiological features, not the aetiology
- Trained predominantly on US and European hospitals; may underperform on images from other populations
- Not validated on mobile/AP X-rays; designed for PA (posteroanterior) films
- GPT-4o-mini reports may generate plausible-sounding but inaccurate clinical statements
- Model outputs sigmoid probabilities, not clinical certainties; threshold selection is not validated

XAI Method

GradCAM (Gradient-weighted Class Activation Mapping) — visualises which spatial regions of the X-ray most strongly activate the class score of interest. Implemented on the final DenseNet dense block. Radiologists should verify the heatmap corresponds to anatomically plausible findings.

Step 13 — Governance Slide (Domain 3)

1. Acceptable Use

 Appropriate contexts:

- Preliminary pathology flagging to support radiologist triage

- Medical education and trainee self-assessment
- Research: feature extraction and baseline comparison for new CXR algorithms

🚫 Explicitly excluded:

- Autonomous reporting without radiologist oversight
- Paediatric patients (not trained or validated on paediatric films)
- Any setting where output is acted on without clinical review
- Use as primary diagnostic tool in emergency settings without human confirmation

2. Data Privacy

- Model weights are open source; no patient data required to run inference
- Uploaded X-rays are processed locally in Colab; no images are sent to external servers
- GPT-4o-mini receives only numeric probability scores — **no pixel data, no patient identifiers**
- Any NHS deployment must satisfy NHS DSPT, GDPR Article 9 (special category health data), and local IG policy

3. Monitoring Outputs

- Monthly radiologist audit: 20 random predictions reviewed against ground truth
- Automated quarterly re-evaluation against a fixed hold-out set; alert if AUC drops > 0.05
- User feedback mechanism: "Was this classification correct?" captured per report
- Retraining/update trigger: if pneumonia sensitivity falls below 0.80 in audit

4. Compliance with Existing Legislation

- **MHRA Software as a Medical Device (SaMD):** likely Class IIa or IIb; regulatory approval required before NHS clinical use
- **NHS AI Framework:** human oversight preserved; model is advisory only; clinician is decision-maker
- **Royal College of Radiologists AI guidance:** AI outputs must be presented as decision support, not diagnosis
- **UK GDPR Article 22:** automated decision-making with significant effects on persons requires human review

5. Ownership & Liability

- If a clinician acts on an incorrect AI prediction and patient harm results: current UK law places liability with **the clinician**, not the model developer or deploying institution

- Developer duty: ensure the system is deployed only within its validated clinical scope
- Escalation pathway: AI output contradicts clinical impression → second radiologist opinion required (not AI re-run)
- Indemnity: NHS trusts deploying AI tools must check their clinical negligence indemnity covers AI-assisted decisions

Demo Day — Judging Criteria

Judges supply **10 live chest X-rays** not seen during evaluation. Your app must be running and ready.

What the judges are assessing

*"Does this AI communicate its findings in a way a clinician could act on?
You are effectively reading a radiology report generated by a machine
— apply the same standards you would to any radiology output." —
Judges' Briefing, Challenge 4*

Scoring Grid

Domain	Criterion
Performance (Slide 1)	Classification accuracy — 10 live images × 1 pt (pneumonia images scored first; missed p
Performance (Slide 1)	GPT radiology report quality — 5 judge-selected images × 1 pt (accurate finding + appro
Explainability (Slide 2)	Model Card /2 · Data Provenance /1 · Known Limitations /1 · XAI Method proposed /1
Governance (Slide 3)	Acceptable Use · Data Privacy · Monitoring · Compliance · Ownership & Liability (1 pt eac
Total	

XAI Judge Answer Key

The expected answer is **GradCAM** (or SHAP / LIME image variant). Your Slide 2 must name the method, show a heatmap from at least one image, and explain *why a radiologist would find it meaningful* — i.e. it reveals **where** the model looked, not just **what** it predicted.